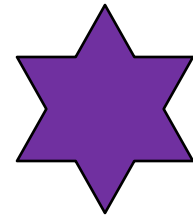
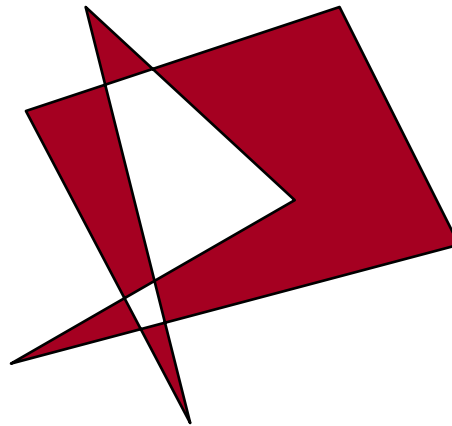
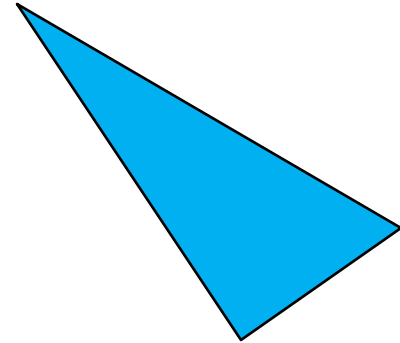
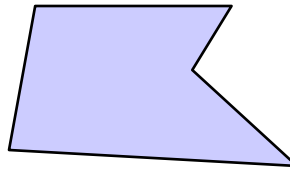
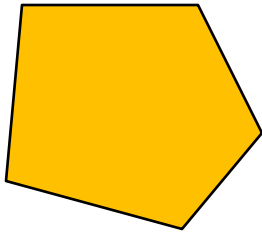
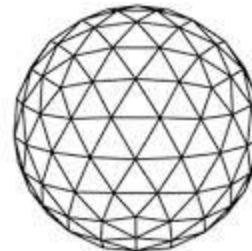
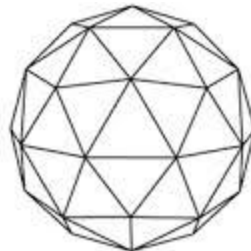
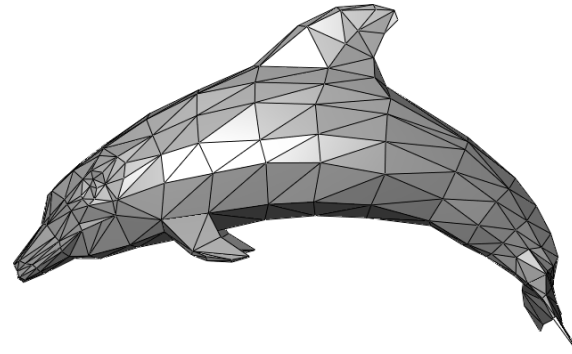
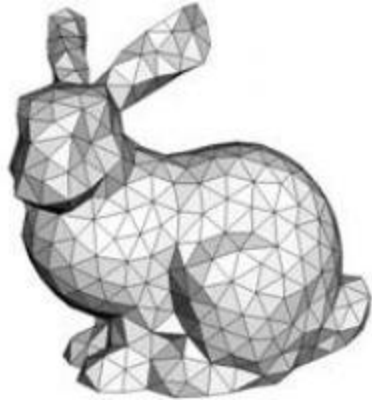


Polygons

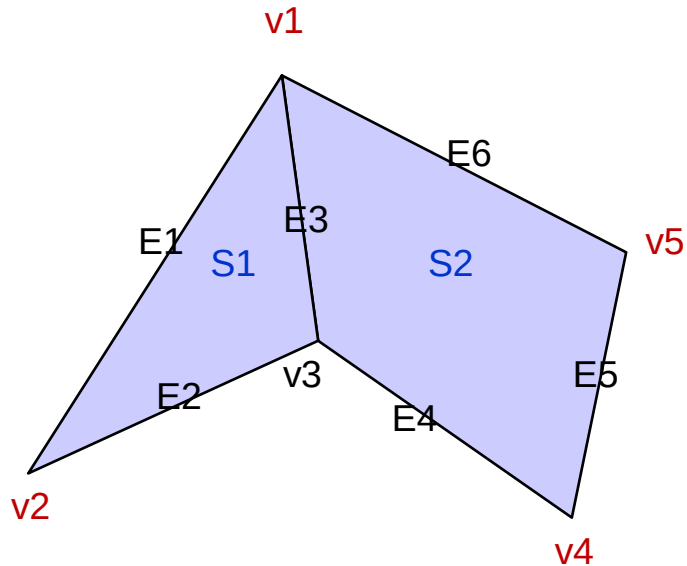
Polygons



Triangle Meshes



Polygon Tables



Vertex Table

v1: x1, y1, z1
v2: x2, y2, z2
v3: x3, y3, z3
v4: x4, y4, z4
v5: x5, y5, z5

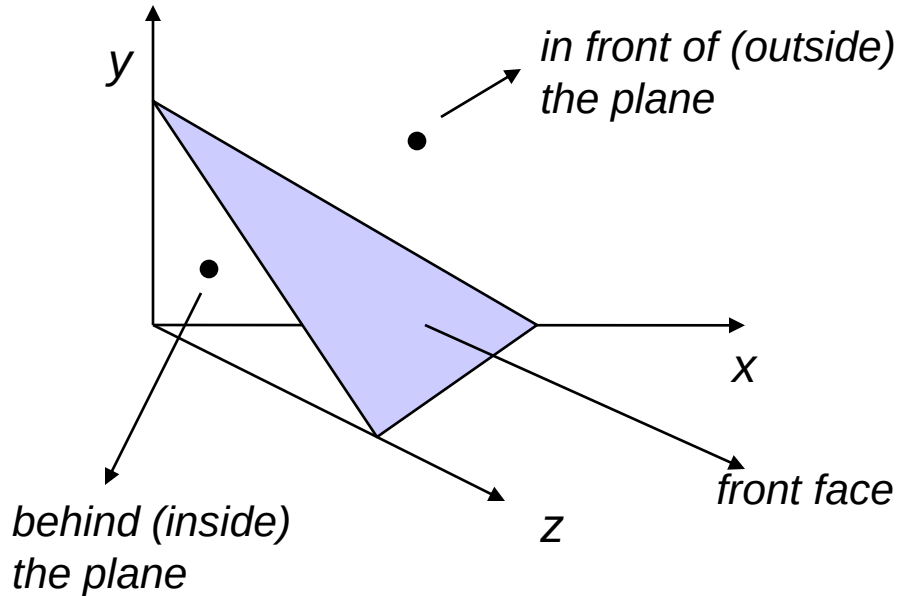
Edge Table

E1: v1, v2
E2: v2, v3
E3: v1, v3
E4: v3, v4
E5: v4, v5
E6: v5, v1

Surface Table

S1: E1, E2, E3
S2: E3, E4, E5, E6

Front and Back Faces



Any point (x, y, z) is:

on the plane:

$$A.x + B.y + C.z + D = 0$$

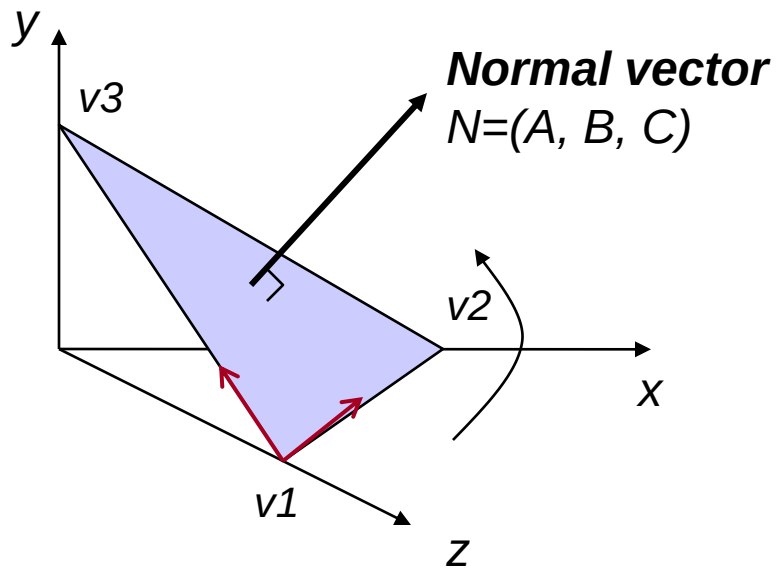
behind the plane:

$$A.x + B.y + C.z + D < 0$$

in front of the plane:

$$A.x + B.y + C.z + D > 0$$

Front and Back Faces



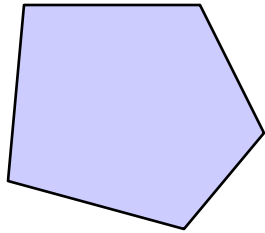
$$A.x + B.y + C.z + D = 0$$

$$\vec{N} = (\vec{v2}-\vec{v1}) \times (\vec{v3}-\vec{v1})$$

$$\vec{N} = \begin{bmatrix} A \\ B \\ C \end{bmatrix}$$

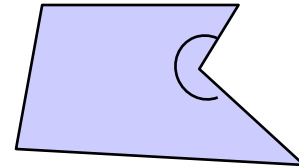
$$\vec{N} \cdot \vec{P} = -D$$

Polygons



convex

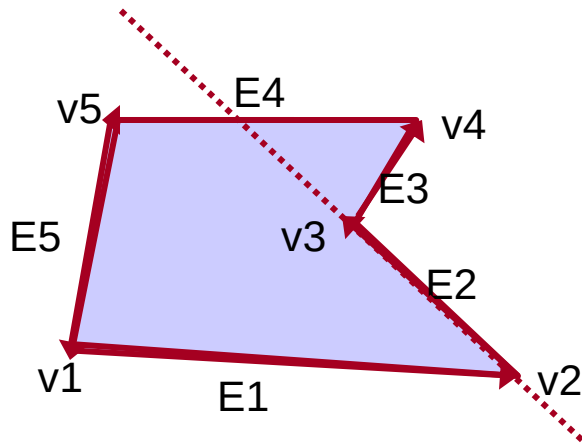
*all interior angles are
 $<180^0$*



concave

*at least one angle is
 $>180^0$*

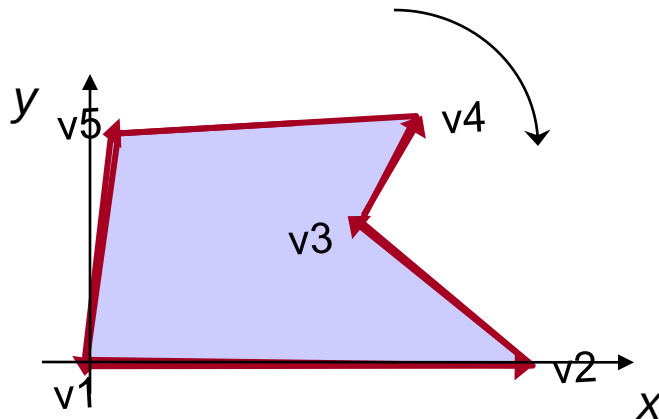
Identifying Concave Polygons



Method 1

if some vertices are on one side and some on the other side of an extension line, then concave

Splitting Concave Polygons



Rotational Method

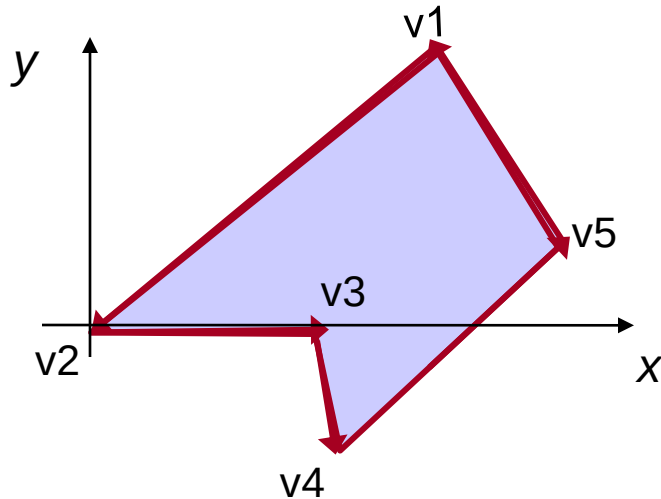
Shift polygon so that each vertex v_k in turn is at the coordinate origin

Rotate polygon about the origin in clockwise direction so that next vertex v_{k+1} is on the x axis

If v_{k+1} is below the x-axis, polygon is concave

Split polygon along x-axis

Splitting Concave Polygons



Rotational Method

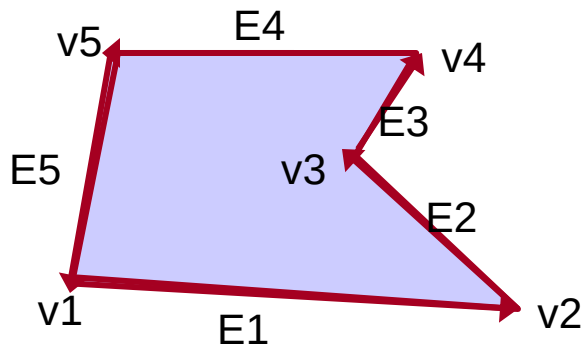
Shift polygon so that each vertex v_k in turn is at the coordinate origin

Rotate polygon about the origin in clockwise direction so that next vertex v_{k+1} is on the x axis

If v_{k+1} is below the x-axis, polygon is concave

Split polygon along x-axis

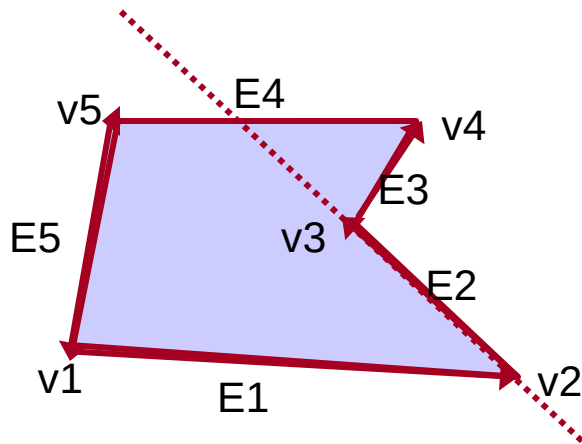
Identifying Concave Polygons



Method 2

if cross product of adjacent vector pairs is < 0 , then concave

Splitting Concave Polygons



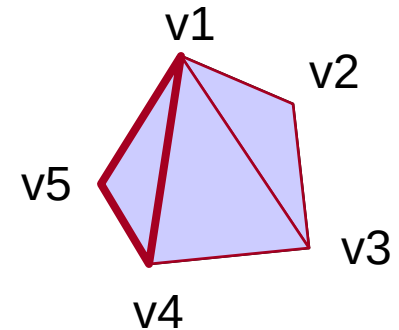
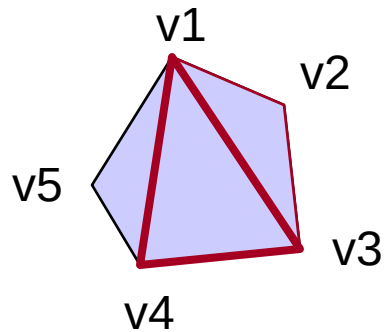
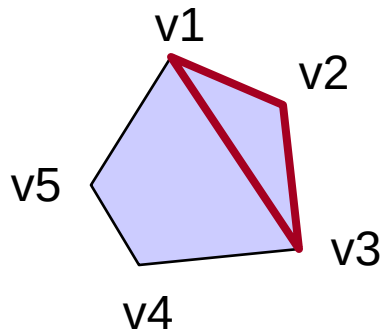
Vector Method

Find the **cross products** of edge vectors ($E_k \times E_{k+1}$)

If **z component is negative** then split polygon along the line of the first edge vector in the cross product pair

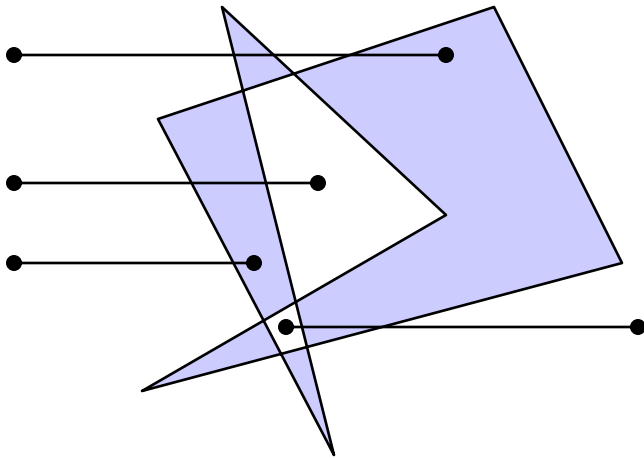
Then find the **intersection** of the line with other edges of the polygon

Splitting Convex Polygons into Triangles



Polygon Filling: Inside-Outside Test

Odd-Even Test



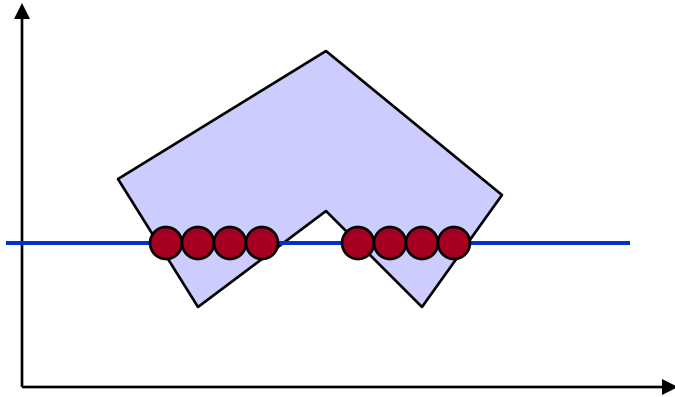
Draw a line from any point P to a point outside the closed polyline

If the number of line-segment crossings is:

odd $\Rightarrow$ P is an *interior* point

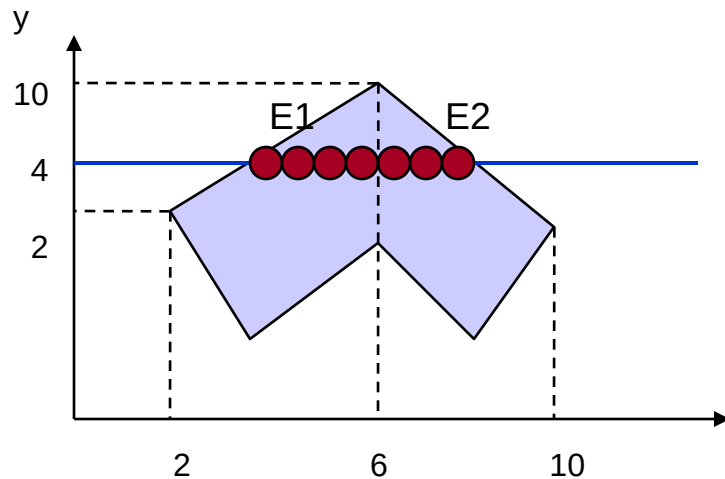
even $\Rightarrow$ P is an *exterior* point

Polygon Filling: Scan-Line Algorithm



- Find **intersection** of polygon boundaries with screen scan lines (solving the pair of simultaneous linear equations where scanline is y , which is a constant)
- Apply fill-color to scan line sections that lie within the fill region (use odd-even rule)

Polygon Filling: Scan-Line Algorithm



E1

$$m = \frac{y_{end} - y_0}{x_{end} - x_0} = \frac{10 - 2}{6 - 2} = 2$$

$$b = y_0 - m.x_0 = -2$$

$$y = 2.x - 2$$

At scanline $y=4$:

E1

$$x = \frac{y + 2}{2} = \frac{4 + 2}{2} = 3$$

$x < 3$ outside (even)

$3 < x < 9$ interior (odd)

$9 < x$ outside (even)

E2

$$m = \frac{10 - 2}{6 - 10} = -2$$

$$b = 22$$

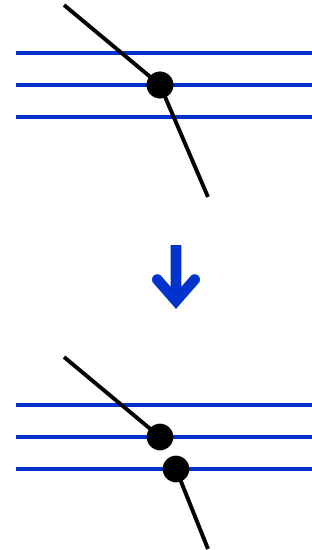
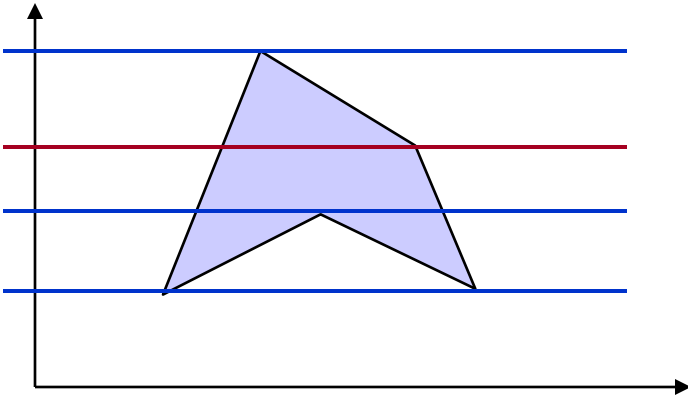
$$y = -2.x + 22$$

E2

$$x = \frac{22 - y}{2} = 9$$

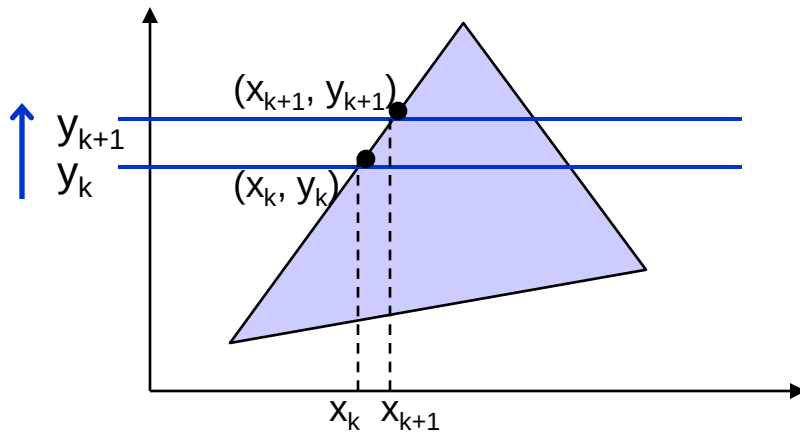
Polygon Filling: Scan-Line Algorithm

- Monotonically increasing/decreasing



Polygon Filling: Scan-Line Algorithm

- Incremental calculation of intersection points

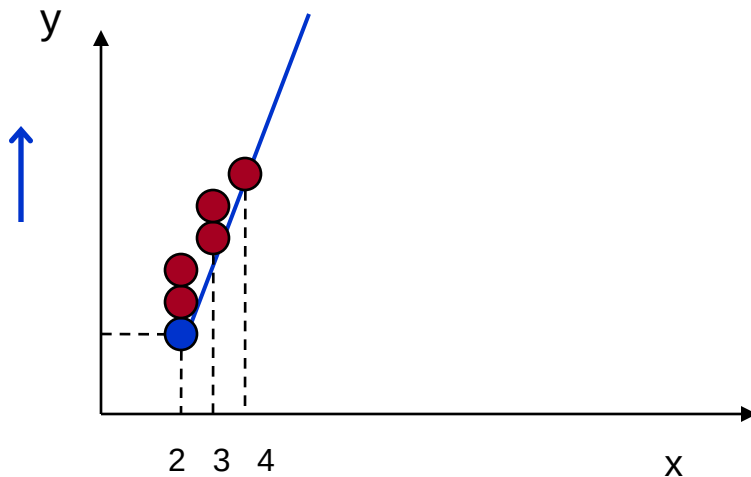


$$m = \frac{y_{k+1} - y_k}{x_{k+1} - x_k} = \frac{1}{x_{k+1} - x_k}$$

$$x_{k+1} = x_k + \frac{1}{m}$$

Polygon Filling: Scan-Line Algorithm

- Integer-increment scheme



$$m = \frac{5}{2}$$

$$\frac{1}{m} = \frac{2}{5}$$

$$x_0 = 2$$

$$x_1 = x_0 + \frac{2}{5} \Rightarrow 2 \quad \text{Truncate} \quad \text{Same}$$

$$x_2 = x_0 + \frac{4}{5} \Rightarrow 2 \quad \text{Same}$$

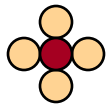
$$x_3 = x_0 + \frac{6}{5} \Rightarrow 3 \quad \text{Increment}$$

$$x_4 = x_3 + \frac{3}{5} \Rightarrow 3 \quad \text{Same}$$

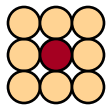
$$x_5 = x_3 + \frac{5}{5} \Rightarrow 4 \quad \text{Increment}$$

$$x_6 = x_5 + \frac{2}{5} \Rightarrow 4$$

Polygon Filling: Boundary-Fill Algorithm

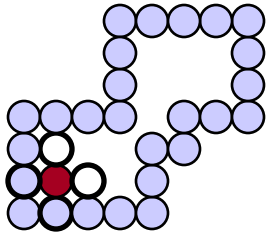


4-connected



8-connected

Polygon Filling: Boundary-Fill Algorithm



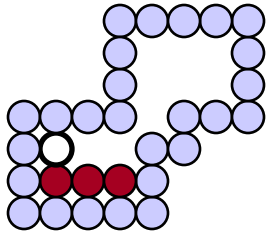
```
void boundaryFill4 (int x, int y, int fillColor, int
    borderColor)
{
    int interiorColor;

    /* Set current color to fillColor, then perform
       following operations. */

    getPixel (x, y, interiorColor);

    if ((interiorColor != borderColor) &&
        (interiorColor != fillColor))
    {
        setPixel (x, y);      // Set pixel color to fillColor
        boundaryFill4 (x + 1, y , fillColor, borderColor);
        boundaryFill4 (x - 1, y , fillColor, borderColor);
        boundaryFill4 (x , y + 1, fillColor, borderColor);
        boundaryFill4 (x , y - 1, fillColor, borderColor)
    }
}
```

Polygon Filling: Boundary-Fill Algorithm



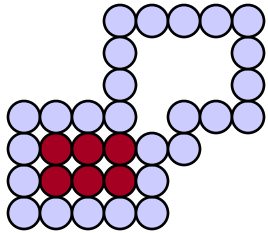
```
void boundaryFill4 (int x, int y, int fillColor, int
    borderColor)
{
    int interiorColor;

    /* Set current color to fillColor, then perform
    following operations. */

    getPixel (x, y, interiorColor);

    if ((interiorColor != borderColor) &&
        (interiorColor != fillColor))
    {
        setPixel (x, y);      // Set pixel color to fillColor
        boundaryFill4 (x + 1, y , fillColor, borderColor);
        boundaryFill4 (x - 1, y , fillColor, borderColor);
        boundaryFill4 (x , y + 1, fillColor, borderColor);
        boundaryFill4 (x , y - 1, fillColor, borderColor)
    }
}
```

Polygon Filling: Boundary-Fill Algorithm



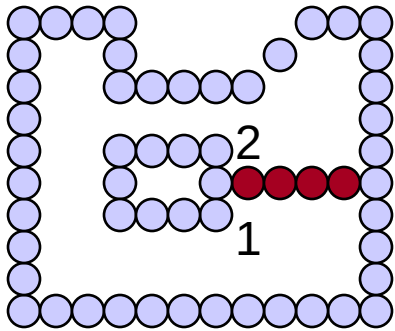
```
void boundaryFill4 (int x, int y, int fillColor, int
    borderColor)
{
    int interiorColor;

    /* Set current color to fillColor, then perform
       following operations. */

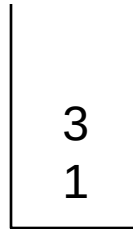
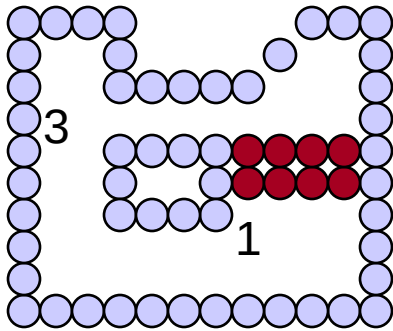
    getPixel (x, y, interiorColor);

    if ((interiorColor != borderColor) &&
        (interiorColor != fillColor))
    {
        setPixel (x, y);      // Set pixel color to fillColor
        boundaryFill4 (x + 1, y , fillColor, borderColor);
        boundaryFill4 (x - 1, y , fillColor, borderColor);
        boundaryFill4 (x , y + 1, fillColor, borderColor);
        boundaryFill4 (x , y - 1, fillColor, borderColor)
    }
}
```

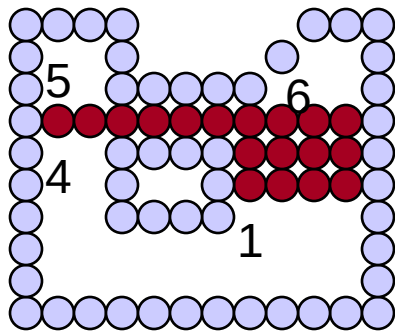
Polygon Filling: Boundary-Fill Algorithm



Polygon Filling: Boundary-Fill Algorithm

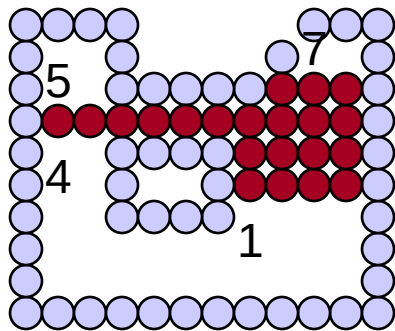


Polygon Filling: Boundary-Fill Algorithm



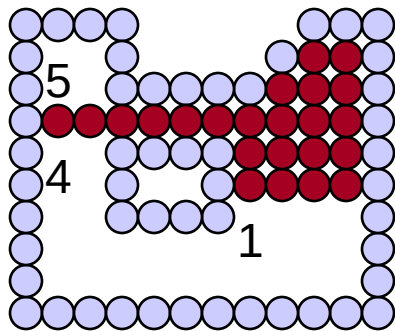
6
5
4
1

Polygon Filling: Boundary-Fill Algorithm



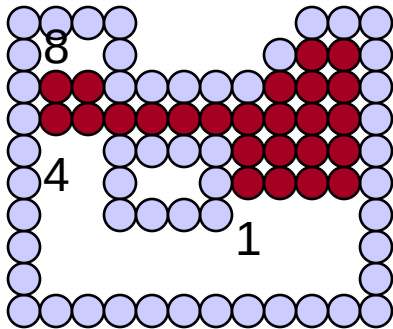
7
5
4
1

Polygon Filling: Boundary-Fill Algorithm



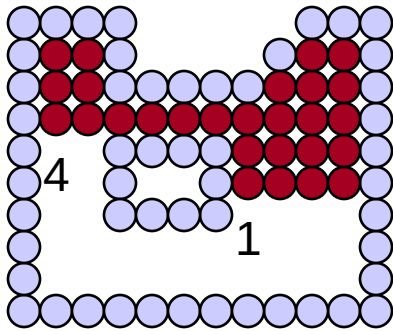
5
4
1

Polygon Filling: Boundary-Fill Algorithm

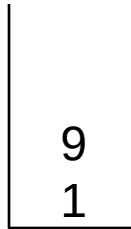
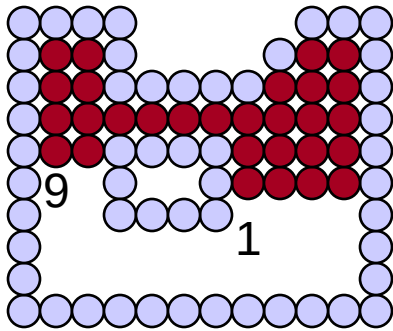


8
4
1

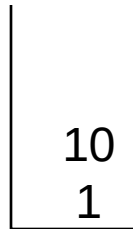
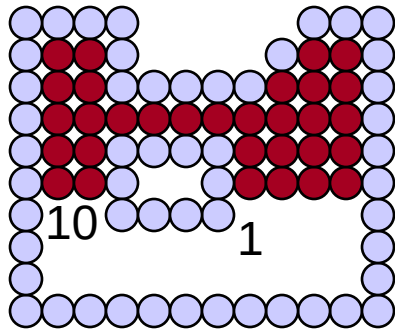
Polygon Filling: Boundary-Fill Algorithm



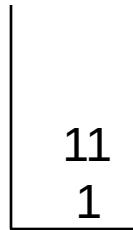
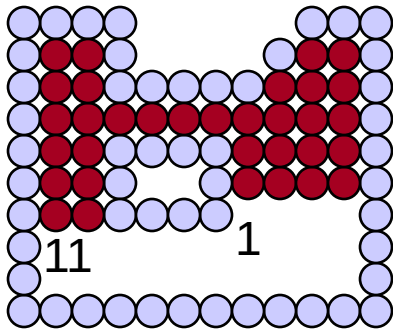
Polygon Filling: Boundary-Fill Algorithm



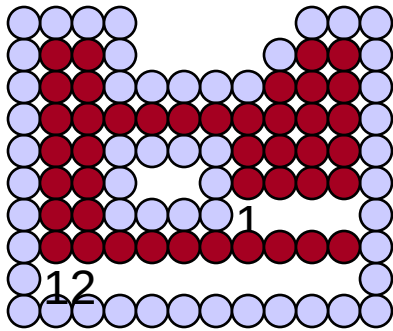
Polygon Filling: Boundary-Fill Algorithm



Polygon Filling: Boundary-Fill Algorithm

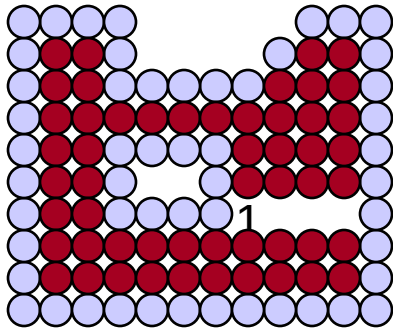


Polygon Filling: Boundary-Fill Algorithm

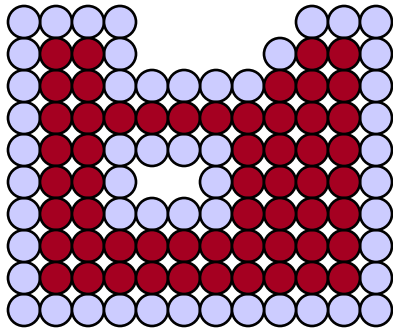


12
1

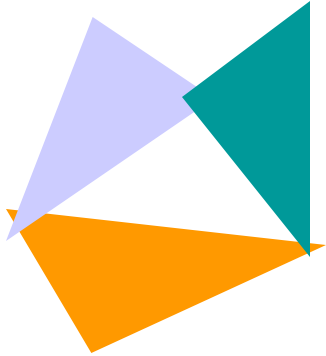
Polygon Filling: Boundary-Fill Algorithm



Polygon Filling: Boundary-Fill Algorithm



Polygon Filling: Flood-Fill Algorithm



```
void floodFill4 (int x, int y, int fillColor, int
    interiorColor)
{
    int color;

    /* Set current color to fillColor, then perform
    following operations. */

    getPixel (x, y, color);

    if (color == interiorColor) {
        setPixel (x, y);    // Set pixel color to fillColor
        floodFill4 (x + 1, y, fillColor, interiorColor);
        floodFill4 (x - 1, y, fillColor, interiorColor);
        floodFill4 (x, y + 1, fillColor, interiorColor);
        floodFill4 (x, y - 1, fillColor, interiorColor)
    }
}
```

OpenGL

Polygon Functions

`glRect* (x1, y1, x2, y2);`

- i (integer)
- s (short)
- f (float)
- d (double)
- v (vector)

Ex:

`glRecti (50, 250, 200, 100);`

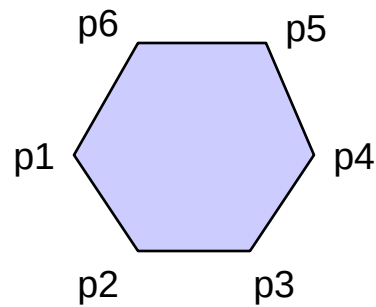


```
int v1[ ]={50, 250};  
int v2[ ]={200, 100};  
glRectiv(v1, v2);
```

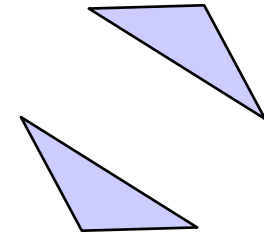
OpenGL

Ex:

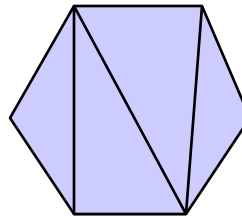
```
glBegin (GL_POLYGON);  
    glVertex2iv(p1);  
    glVertex2iv(p2);  
    glVertex2iv(p3);  
    glVertex2iv(p4);  
    glVertex2iv(p5);  
    glVertex2iv(p6);  
glEnd();
```



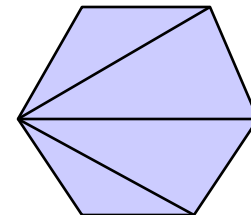
GL_POLYGON



GL_TRIANGLES



GL_TRIANGLE_STRIP

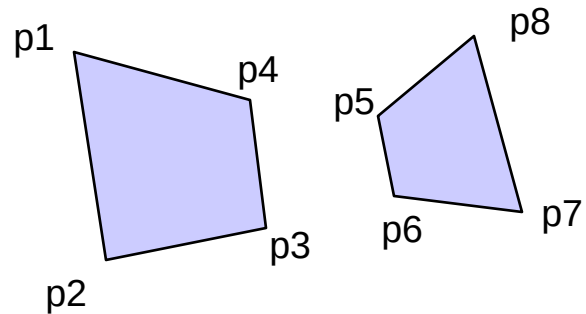


GL_TRIANGLE_FAN

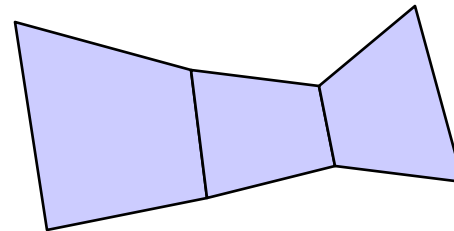
OpenGL

Ex:

```
glBegin (GL_QUADS);  
  glVertex2iv(p1);  
  glVertex2iv(p2);  
  glVertex2iv(p3);  
  glVertex2iv(p4);  
  glVertex2iv(p5);  
  glVertex2iv(p6);  
  glVertex2iv(p7);  
  glVertex2iv(p8);  
glEnd();
```



GL_QUADS



GL_QUAD_STRIP